Frontend-Backend Connection Contract and System Evolution Analysis

Intelligent IoT Data Management Project

Abstract

The Intelligent IoT Data Management project is evolving from a prototype-style IoT dashboard into a more structured web application with separate frontend and backend responsibilities. The current project contains a new backend implementation under newBackend and a new frontend implementation under new-frontend/frontend. Although both sides show progress, the project still needs a clear frontend–backend connection contract so that the frontend team can connect dashboard hooks and components to backend APIs without relying on assumptions.

This research analyses the current project structure, identifies integration gaps between newBackend and new-frontend/frontend, and proposes a practical connection contract for frontend developers. The contract defines endpoint usage, request formats, response formats, error handling, frontend/backend responsibilities and future testing recommendations. Recent frontend–backend integration research highlights that standardised API interfaces, documentation, formal contracts, testing and monitoring help reduce inconsistencies between client-side and server-side components (Bogutskii 2025:111–13). API-first and API-led integration approaches are also recommended in modern enterprise systems because they improve system connectivity, maintainability and development efficiency (Jayaprakash, 2025).

The main contribution of this research is a project-specific connection contract that can guide the frontend team when replacing mock/local data flows with backend-driven dataset-based API calls.

Contents

1. Introduction

Effective communication between frontend dashboards and backend data services is a foundational requirement of modern IoT application development. Within this two-layer architecture, the frontend governs user-facing concerns interaction design, data visualization, filtering controls, charts, and feedback messages while the backend manages the data lifecycle: storage, retrieval, validation, transformation, and API-based delivery. When these two layers operate without a clearly defined shared contract, integration failures can emerge even when each layer functions correctly in complete isolation.

The Intelligent IoT Data Management project illustrates this challenge precisely. The project now includes a restructured backend directory, new Backend, alongside a redesigned frontend directory, new frontend/frontend. On the backend side, new API routes expose dataset-driven endpoints covering dataset metadata, time-series data, filtered series, and timestamp queries. On the frontend side, the dashboard is built around modular, reusable hooks useSensorData, useStreamNames, useFilteredData, and useTimeRange that encapsulate stateful logic. This hook-based architecture is inherently well-suited to integration work, as custom hooks enable reusable stateful logic to be shared cleanly across components (React, 2025).

Despite this structural readiness, a critical misalignment persists. The frontend continues to rely on mock and local data in key areas, despite the backend already providing dataset-based routes capable of supporting real-time data retrieval and filtering. A concrete example highlights this gap: the frontend dashboard currently invokes use Sensor Data(true), which locks the dashboard into mock mode. Internally, the useSensorData.js hook imports sensor Data. Json for this mock mode. when mock mode is disabled, it attempts to contact /api/sensor-data a route that does not exist in the current backend structure. The backend instead exposes routes such as /api/ datasets, /api/datasets/:name/series,/api/datasets/:name/series/filter, and /api/datasets/:name/timestamps.

This disconnect reveals that the project's core challenge is not a shortage of code, but rather the absence of a formally agreed frontend–backend connection contract. Such a contract defines the complete terms of integration: how the frontend should invoke backend endpoints, what request data should be transmitted, what response format should be expected, and how errors should be consistently identified and handled.

A design-first API methodology is particularly well-suited to resolving this kind of structural ambiguity, as it encourages teams to identify concrete use cases upfront, engage relevant stakeholders, produce explicit API contract definitions, and enforce consistent conventions around HTTP status codes, versioning, and error responses (Microsoft, 2024).

This research therefore proposes a practical, evidence-grounded connection contract for the Intelligent IoT Data Management project. The objective is twofold: to make backend integration meaningfully clearer for frontend developers, and to ensure that every design decision remains anchored in the actual structure and behavior of the existing project.

2. Research Aim and Scope

The aim of this research is to develop a frontend–backend connection contract for the Intelligent IoT Data Management project, based on the current newBackend and new-frontend/frontend implementation.

The research focuses on four objectives:

1. Analyze the current frontend and backend integration state.
2. Identify project-specific mismatches between frontend assumptions and backend APIs.
3. Propose a clear dataset-based connection contract for frontend developers.
4. Justify the contract using project evidence and recent API integration literature.

The scope is intentionally focused. This research does not attempt to redesign the whole IoT platform or discuss unrelated IoT theory. Instead, it focuses on the practical integration layer between the frontend dashboard and backend API routes. The previous research direction identified the importance of a frontend–backend contract, but this version gives higher priority to the current dataset-based backend architecture because that is the clearer long-term integration path for the project.

3. Methodology

This research uses a qualitative system analysis approach. The analysis is based on three types of evidence.

First, the current project zip was reviewed, with priority given to newBackend and new frontend/frontend. The older backend and frontend folders are treated only as historical or prototype evidence. The current system evidence comes mainly from:

- newBackend/src/server.js
- newBackend/src/routes/index.js
- newBackend/src/routes/datasetRoutes.js
- newBackend/src/routes/seriesRoutes.js
- newBackend/src/routes/timestampsRoutes.js
- newBackend/src/controllers/datasetsController.js
- newBackend/src/controllers/seriesController.js
- newBackend/src/controllers/timestampsController.js
- newBackend/src/controllers/analyseController.js

- newBackend/src/services/timeseriesService.js
- newBackend/src/repositories/timeseriesRepository.js
- new-frontend/frontend/src/components/Dashboard.jsx
- new-frontend/frontend/src/hooks/useSensorData.js
- new-frontend/frontend/src/hooks/useStreamNames.js
- new-frontend/frontend/src/hooks/useFilteredData.js
- new-frontend/frontend/src/hooks/useTimeRange.js

Second, the previous research draft was reviewed to understand the original direction of the work. The previous version already identified the importance of a frontend–backend contract, but it included both simple stream routes and dataset-based routes. This corrected version treats the dataset-based controller structure as the preferred backend integration path.

Third, recent literature and technical guidance from 2024 onwards were used to support the recommendations. Recent research on frontend and backend integration identifies data format inconsistency, API documentation, formal contracts, testing and monitoring as important concerns in web application integration (Boguski 2025).

Recent API integration literature also supports API-led integration, RESTful API usage, documentation, testing and monitoring in modern application environments (Jayaprakash 2025). Microsoft's API design guidance further supports contract-first API design and consistent API behavior (Microsoft 2024).

4. Current Project Evidence

The current project shows that the frontend and backend are moving in the right direction, but they are not fully connected yet.

4.1 Backend evidence from newBackend

The backend uses an Express server in newBackend/src/server.js. It enables JSON request handling and mounts API routes under /api. This means frontend API calls should be designed around the /api route prefix.

The current backend route structure shows a dataset-based API direction. Instead of relying on generic stream routes, the newer backend separates dataset metadata, dataset series, dataset filtering and timestamp retrieval into specific controllers. This is a cleaner architecture because each endpoint has a clear purpose and the frontend can request data for a selected dataset.

Backend route	Method	Purpose
/api/datasets	GET	Returns available dataset metadata
/api/datasets/:id	GET	Returns one dataset by ID
/api/datasets/:name/series	GET	Returns time-series entries for a selected dataset
/api/datasets/:name/series/filter	POST	Filters selected stream names within a dataset
/api/datasets/:name/timestamps	GET	Returns timestamps for time range controls
/api/analyse	POST	Supports future analysis or insight requests

The backend also contains a database-oriented structure. timeseriesRepository.js works with datasets and timeseries long, while timeseriesService.js converts long-format time-series rows into wide-format entries. This is important because the frontend charts need wide-format entries such as:

```
{
  "created_at": "2025-03-19T15:01:59.000Z",
  "entry_id": 3242057,
  "Temperature": 22,
  "RHHumidity": 45
}
```

The backend therefore already provides a useful foundation for frontend integration. The main issue is that the frontend needs a clear guide for which dataset-based routes to call and what shape of data to expect.

4.2 Frontend evidence from new frontend/frontend

The current frontend already has a dashboard and reusable hooks. This is a positive direction because it separates data and filtering logic from the visual components. The main dashboard is in new-frontend/frontend/src/components/Dashboard.jsx.

The frontend currently uses:

```
const {data, loading, error } = useSensorData(true);
```

The true value means the dashboard is using mock mode. In useSensorData.js, the hook imports local mock data from:

```
../data/sensorData1.json
```

When mock mode is disabled, the hook attempts to call:

```
fetch('/api/sensor-data')
```

However, this route does not match the preferred dataset-based backend route structure in newBackend. Instead of calling /api/sensor-data, the frontend should move toward dataset-based API calls.

The frontend also derives stream names locally in useStreamNames.js by reading object keys from the first data entry and removing entry_id and created_at. This works with static data, but it is less reliable for backend-driven data because the available stream names should depend on the selected dataset.

The frontend filters data locally in useFilteredData.js by time range, entry ID and selected streams. This is useful for prototype behaviour, but backend filtering already exists through /api/datasets/:name/series/filter. For larger IoT datasets, backend filtering is more scalable because the server can reduce the amount of data sent to the browser.

This project evidence shows that the frontend and backend are close to integration, but they need a shared contract to remove mismatched assumptions.

5. Identified Integration Gap

The main integration gap is the absence of a documented connection contract between new frontend/frontend and newBackend.

The current frontend assumes:

- data can be loaded from local JSON;
- API mode can call /api/sensor-data.
- stream names can be derived from the first data row;
- filtering can happen mainly in the browser;
- selected stream fields can be handled locally.

The current backend provides:

- /api/datasets;
- /api/datasets/:name/series;
- /api/datasets/:name/series/filter;
- /api/datasets/:name/timestamps;
- /api/analyse;
- a service/repository pattern for database-backed time-series data;
- wide-format transformation through the service layer;
- dataset-based filtering support.

These are not impossible to connect, but they require agreed rules. Data format inconsistencies are a recognised frontend–backend integration challenge, and detailed API documentation with standard formats such as JSON can reduce these errors (Bogutskii 2025). In this project, the frontend still behaves as if data can be loaded from mock/local data or a generic /api/sensor-data route, while the backend has moved toward dataset-specific routes. Therefore, the frontend needs a contract that explains how to select a dataset, load that dataset's series, filter streams inside that dataset and retrieve timestamps for time range controls.

6. Proposed Frontend–Backend Connection Contract

This section is the main output of the research. It defines how the frontend should connect to the backend.

6.1 Contract purpose

The purpose of the contract is to create a shared agreement between frontend and backend developers. The contract should answer these questions:

- Which endpoint should the frontend call?
- Which HTTP method should be used?
- What request body should be sent?
- What response format should be expected?
- What error format should the frontend handle?
- Which layer is responsible for filtering, validation and visualisation?

OpenAPI describes a standard, language-agnostic interface for HTTP APIs so that humans and systems can understand service capabilities without needing to inspect source code or network traffic (OpenAPI Initiative 2024a). This supports the idea that the project should document API behavior clearly before deeper integration continues.

6.2 Consumer and provider roles

Role	Project component	Responsibility
API consumer	new frontend/frontend	Calls backend endpoints, sends selected filter values, displays charts and user messages
API provider	newBackend	Provides routes, validates input, retrieves/filters data and returns structured JSON responses

This separation is important because the frontend should not need to understand database tables such as timeseries_long, and the backend should not need to control how charts are displayed. RESTful API design guidance also recommends APIs that are easy to understand, flexible and maintainable (Microsoft Azure Architecture Center 2025).

6.3 Endpoint mapping for frontend developers

Frontend need	Current frontend area	Backend endpoint	Method	Recommendation
Load available datasets	Future dataset selector	/api/datasets	GET	Use this to populate dataset choices
Load one dataset's time-series data	useSensorData(datasetName)	/api/datasets/:name/series	GET	Main backend-driven data loading route
Filter selected streams in a dataset	useFilteredData(datasetName, streamNames) or submit handler	/api/datasets/:name/series/filter	POST	Send selected streams using streamNames
Load timestamps for selected dataset	useTimeRange(datasetName)	/api/datasets/:name/timestamps	GET	Use this for time range dropdowns
Send future analysis request	Future analysis or insight feature	/api/analyse	POST	Use later for backend-based insights

GET should be used for retrieving data, while POST should be used when the frontend sends selected values to the backend for processing. The GET method requests a representation of a resource, while the POST method sends data to the server (MDN Web Docs 2025a; MDN Web Docs 2025b).

7. Recommended Request and Response Formats

7.1 Load available datasets

Endpoint

```
GET /api/datasets
```

Expected frontend use

Use this when the frontend needs to show available datasets in a dataset selector.

Expected response

```
[
 {
  "id": 1,
  "name": "sensor1"
 }
]
```

Project justification

The backend has moved toward dataset-based routes. Therefore, the frontend should first know which datasets are available before requesting series data. This avoids assuming that there is only one global dataset.

7.2 Load dataset series

Endpoint

```
GET /api/datasets/:name/series
```

Example

```
GET /api/datasets/sensor1/series
```

Expected response

```
[
 {
  "created_at": "2025-03-19T15:01:59.000Z",
  "entry_id": 3242057,
  "Temperature": 22,
  "RH Humidity": 45
 }
]
```

Project justification

The backend timeseriesService.js converts long-format database rows into wide-format entries. This is useful because frontend charts can continue using a wide format without understanding the database structure.

7.3 Filter dataset series

Endpoint

```
POST /api/datasets/:name/series/filter
```

Example

```
POST /api/datasets/sensor1/series/filter
```

Required request body

```
{
 "streamNames": ["Temperature", "RH Humidity"]
}
```

Expected response

```
[
 {
  "created_at": "2025-03-19T15:01:59.000Z",
  "entry_id": 3242057,
  "Temperature": 22,
  "RH Humidity": 45
 }
]
```

Project justification

This route is suitable for the long-term system because it connects filtering to a named dataset. This is better than assuming only one global stream dataset exists. It also makes the contract clearer because the frontend must send selected streams using the exact field name streamNames.

7.4 Load timestamps for time range controls

Endpoint

```
GET /api/datasets/:name/timestamps
```

Example

```
GET /api/datasets/sensor1/timestamps
```

Expected response

```
[
  "2025-03-19T15:01:59.000Z",
  "2025-03-19T15:02:29.000Z"
]
```

Project justification

The current frontend useTimeRange.js derives timestamps from already-loaded data. This works in mock mode, but it means the frontend must load the full dataset before it can populate time range controls. A backend timestamp route can support cleaner time range selection.

7.5 Future analysis request

Endpoint

```
POST /api/analyse
```

Expected frontend use

Use this later when the frontend sends selected data or selected stream information for backend-based analysis.

Project justification

The current frontend performs many analysis and visualisation tasks locally. The /api/analyse route provides a possible future direction for moving advanced insights, statistics or analysis logic into the backend.

8. Error Handling Contract

The frontend must handle backend errors in a predictable way, especially when using dataset-based routes.

8.1 Missing or empty streamNames field

Scenario

The frontend sends an empty request body or sends streamNames as an empty array.

Expected status

400 Bad Request

Expected response

```
{
  "error": "streamNames must be a non-empty array"
}
```

Frontend behavior

The frontend should prevent submission when no stream is selected. The dashboard already includes an empty state when no streams are selected, so this can be extended to block backend filter requests until the selection is valid.

8.2 Invalid stream names for a dataset

Scenario

The frontend sends a stream name that does not exist in the selected dataset.

Expected status

400 Bad Request

Recommended response

```
{
  "error": "Invalid stream names provided",
  "invalidStreamNames": ["wrongField"],
  "availableStreamNames": ["field1", "field2", "field3"]
}
```

Frontend behaviour

The frontend should show a user-friendly message such as:

One or more selected streams are not available for this dataset. Please refresh the stream list and select again.

Project justification

This is important because the current frontend mock data may use names such as field1 and field2, while backend/database data may use metric names such as Temperature or RH Humidity. This mismatch provides direct evidence for why the frontend should not hardcode stream names.

8.3 Dataset not found

Scenario

The frontend requests a dataset name that does not exist.

Expected status

404 Not Found

Expected response

```
{
  "error": "Dataset not found or empty"
}
```

Frontend behaviour

The frontend should show:

Dataset not found or no data is available for this dataset.

8.4 Server error

Scenario

Unexpected backend/database failure.

Expected status

500 Internal Server Error

Expected response

```
{
  "error": "Failed to load series"
}
```

Frontend behaviour

The frontend should show a general error message and avoid crashing the dashboard.

9. Responsibility Separation

A contract is not only about endpoint names. It also defines which side is responsible for each task.

Task	Frontend responsibility	Backend responsibility
Display dashboard layout	Yes	No
Render charts and scatter plots	Yes	No
Show loading/error states	Yes	Return status and error messages
Provide available datasets	No	Yes
Provide dataset time-series data	No	Yes
Validate selected dataset name	Basic UI validation only	Yes
Validate selected stream names	Basic UI validation only	Yes
Filter large datasets	Prefer backend	Yes
Convert DB long format to chart-friendly wide format	No	Yes
Calculate frontend-only visual insights	Yes, where needed	Optional future support
Manage database queries	No	Yes

This separation reduces duplicated logic. It also makes the project easier to maintain because the backend can change its database implementation while keeping the frontend response format stable. Recent frontend–backend integration research identifies formalising contracts between systems as important because it supports consistent data exchange protocols and reduces unnecessary interdependence between components (Bogutskii 2025:112).

10. Contract Implementation Plan

Rather than replacing the existing data layer all at once, the proposed contract should be introduced incrementally. This allows the frontend and backend teams to move toward full integration without disrupting the current working dashboard. Since the frontend currently depends on mock data in several key areas, the first stage should not remove mock mode completely. Instead, the frontend should be refactored to support a dual-mode architecture, where the same codebase can handle both local mock data and live backend API responses.

Step 1: Introduce dataset selection

First, the frontend should introduce a dataset selection flow. A future dataset selector should call /api/datasets to retrieve the available datasets. This allows the frontend to work with selected datasets instead of assuming one global dataset.

Step 2: Update useSensorData to support dataset-based API calls

Second, the useSensorData hook should be updated to accept a dataset name parameter. Currently, the dashboard calls useSensorData(true), which tells the frontend to load data locally from mock sources.

Once backend integration is activated, this hook should send its request to:

```
/api/datasets/:name/series
```

instead of the outdated:

```
/api/sensor-data
```

This change would allow the dashboard to load live backend-driven data dynamically for any selected dataset.

Step 3: Move filtering to the backend when API mode is active

Third, filtering logic should also be migrated carefully. Local filtering can remain as a fallback when the system is running in mock mode. However, when backend mode is active, filtering should be handled by the server.

In that case, the selected stream names must be sent to

```
/api/datasets/:name/series/filter
```

using a POST request. The request body must use the exact field name:

```
{
  "streamNames": ["field1", "field2"]
}
```

The naming and structure should not change, because even a small mismatch can cause frontend-backend integration errors.

Step 4: Update useTimeRange to use backend timestamps

Fourth, the useTimeRange hook should be updated to support backend-driven timestamps. Instead of always deriving timestamps from already-loaded data, the hook should call:

```
/api/datasets/:name/timestamps
```

when a dataset is selected. This would allow the frontend to populate time range controls without loading the full dataset first.

Step 5: Keep the response format stable

Fifth, the response format should remain stable for the frontend. The backend should continue returning wide-format data entries with:

```
created_at, entry_id, and selected stream values.
```

This is important because the current chart components already depend on this structure.

Step 6: Add testing around the contract boundaries

Finally, the integration work should include testing based on the contract. Backend tests should confirm that each dataset route returns responses in the agreed format, so the frontend team knows exactly what to expect.

Frontend tests should confirm that each hook calls the correct endpoint and behaves properly in three key states: loading, successful response, and error. This should include unit, integration, and end-to-end testing, because this combination is widely regarded as important for reliable frontend–backend integration (Bogutskii, 2025).

11. Research Justification

The proposed contract is justified by both project evidence and recent literature.

Examining the project itself, the misalignment between frontend and backend is both specific and consequential. The frontend is equipped with well-structured reusable hooks, yet its current API call to /api/sensor-data bears no correspondence to the backend's actual route architecture. Compounding this, the dashboard remains locked in mock mode, and stream names are resolved entirely through local logic rather than server-side retrieval. On the backend, meanwhile, a dataset-centric API already exists, covering dataset listing, series retrieval, filtered series, and timestamp queries. This gap makes clear that what the project requires is not an abstract or theoretical contract, but a grounded, practical one that draws direct lines between what the frontend currently needs and what the backend already offers.

The research literature lends strong and consistent support to exactly this kind of intervention. Standardised API interfaces are identified as a primary mechanism for achieving reliable frontend–backend connectivity, with formal contracts, API documentation, validation schemas, and versioning collectively improving the transparency and predictability of data exchange between system layers (Bogutskii, 2025:111–12). API integration is further characterised as a core capability of modern connected systems, one that underpins reusable integration components, scalability, structured testing, and ongoing operational monitoring (Jayaprakash, 2025:637–38, 643). Microsoft's API design guidance reinforces this position, recommending that teams produce explicit API contract definitions and enforce consistency across status codes and error responses throughout the development lifecycle (Microsoft, 2024).

12. Future Improvements

The first recommended improvement is to add a dataset-specific stream names endpoint. A useful future endpoint would be:

```
GET /api/datasets/:name/stream-names
```

This would allow the frontend to load available stream names for the selected dataset without first loading the full series data.

The second improvement is to add backend-supported time filtering. The current frontend supports time range filtering locally, and the backend provides timestamps. However, a future backend filter request could support.

```
{
  "streamNames": ["Temperature", "RH Humidity"],
  "startTime": "2025-03-19T15:00:00.000Z",
  "endTime": "2025-03-19T16:00:00.000Z"
}
```

This would reduce browser processing and make the system more scalable for larger IoT datasets.

The third improvement is to create an OpenAPI/Swagger specification for the backend. OpenAPI is designed to describe HTTP APIs so consumers can understand service capabilities without reading source code or inspecting network traffic (OpenAPI Initiative 2024a). OpenAPI 3.1.1 is also described as the recommended version for new projects where teams have the option to adopt it (OpenAPI Initiative 2024b). This would make the contract easier to maintain and could support future automated documentation or testing.

The fourth improvement is to standardise all success responses with metadata. For example:

```
{
  "data": [],
  "metadata": {
    "count": 0,
    "datasetName": "sensor1",
    "streamNames": []
  }
}
```

This is not required immediately because the current frontend expects arrays, but it would make the API more consistent in the future.

13. Conclusion

The Intelligent IoT Data Management project stands at a decisive crossroads in its integration journey. The newBackend already delivers a meaningful foundation dataset-based API routes covering dataset discovery, filtered series retrieval, and timestamp querying are all in place. Equally, the new frontend/frontend arrives with a coherent dashboard architecture and a suite of purpose-built hooks handling data loading, stream name resolution, filtering, and time range management. Yet despite this parallel progress on both sides, the two layers remain structurally disconnected.

The root of this disconnect is not an absence of development effort on either side. Rather, it is the absence of a shared understanding between them. The frontend continues to operate against mock data and an endpoint path that no longer reflects the backend's actual route structure, while the backend has already moved toward a dataset-centric API model. Without a governing contract to bridge this gap, integration remains fragile vulnerable to small but consequential mismatches in endpoint naming, field identifiers, response shapes, and error handling conventions.

To address this, the research proposed a project-specific frontend–backend connection contract tailored directly to the realities of this codebase. The contract codifies endpoint mappings, request and response formats, error handling expectations, and a clear separation of responsibilities between layers. Looking further ahead, it also identifies a roadmap of targeted improvements dataset-specific stream name resolution, server-side time filtering, OpenAPI documentation, and a structured integration testing strategy.

In practical terms, the proposed contract hands the frontend team a clear, self-contained reference for connecting to newBackend one that removes the need to parse unrelated backend implementation details. More broadly, it charts a credible path for the project's evolution: from a mock-data prototype into a maintainable, scalable, and genuinely backend-driven IoT data management platform.

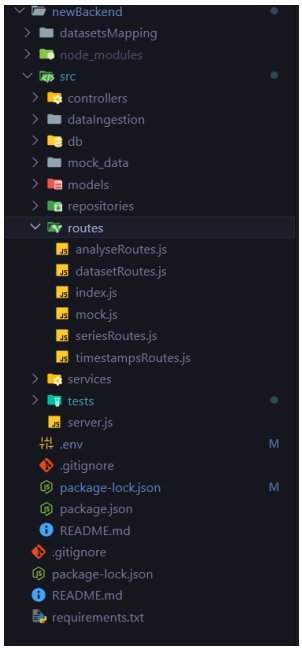
Proposed Frontend-Backend Contract Summary

Frontend requirement	Proposed backend endpoint	Method	Request format	Expected response	Frontend use
Load available datasets	/api/datasets	GET	No request body required	List of available datasets such as sensor1, sensor2, sensor3	Populate dataset selector
Load series data for selected dataset	/api/datasets/:name/series	GET	Dataset name in URL path	Wide-format time-series records with created_at, entry_id, and sensor fields	Display chart and dashboard data
Filter selected streams	/api/datasets/:name/series/filter	POST	{ "streamNames": ["field1", "field2"] }	Filtered records containing only selected stream fields	Allow users to select specific streams/metrics
Load timestamps for selected dataset	/api/datasets/:name/timestamps	GET	Dataset name in URL path	List of available timestamps for the selected dataset	Support time range filtering
Handle invalid filter request	/api/datasets/:name/series/filter	POST	Missing or empty streamNames field	400 Bad Request with validation error	Display validation/error feedback to user

Appendices

Appendix A: Current Backend Route Evidence

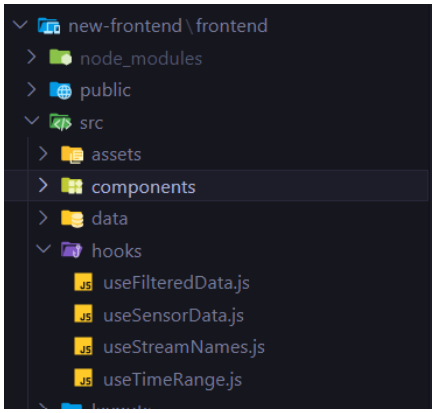
Appendix A shows the current backend route structure in newBackend. The evidence shows that backend routes are mounted under /api and that the backend provides dataset, series, filtering, timestamp and analysis endpoints. This supports the need for a frontend–backend contract because frontend developers need to know which exact API routes should be used for dashboard data loading and filtering.



newBackend/src/server.js
newBackend/src/routes/index.js
newBackend/src/routes/datasetRoutes.js
newBackend/src/routes/seriesRoutes.js
newBackend/src/routes/timestampsRoutes.js
newBackend/src/routes/analyseRoutes.js

Appendix B: Frontend Hook Evidence

Appendix B shows the current frontend hook-based structure in new-frontend/frontend. The dashboard uses reusable hooks for loading sensor data, stream names, filtered data and time range values. This supports the proposed contract because these hooks can become the main connection points between the frontend dashboard and backend API endpoints.



new-frontend/frontend/src/components/Dashboard.jsx
new-frontend/frontend/src/hooks/useSensorData.js
new-frontend/frontend/src/hooks/useStreamNames.js
new-frontend/frontend/src/hooks/useFilteredData.js
new-frontend/frontend/src/hooks/useTimeRange.js

Appendix C: Mock Data and API Mismatch Evidence

Appendix C shows the current mismatch between frontend API assumptions and backend route availability. The frontend dashboard currently uses useSensorData(true), which enables mock mode. The useSensorData.js hook also imports local data from sensorData1.json and attempts to call /api/sensor-data when API mode is used.

However, the current backend's preferred path uses dataset-based routes such as /api/datasets, /api/datasets/:name/series. This mismatch provides direct project evidence for why a frontend–backend connection contract is required.

- new-frontend/frontend/src/components/Dashboard.jsx
- new-frontend/frontend/src/hooks/useSensorData.js
- newBackend/src/routes/datasetRoutes.js
- newBackend/src/routes/seriesRoutes.js
- newBackend/src/routes/timestampsRoutes.js

Appendix C.1- Dashboard currently enables mock data mode

new-frontend/frontend/src/components/Dashboard.jsx

```
const Dashboard = () => {
  const { data, loading, error } = useSensorData(true); // mock mode
  const streamNames = useStreamNames(data);
```

Appendix C.2 - Frontend hook imports local mock JSON data

new-frontend/frontend/src/hooks/useSensorData.js

```
// hooks/useSensorData.js
import { useState, useEffect } from 'react';
import SensorData1 from '../data/sensorData1.json';
```

Appendix C.3-Frontend expects /api/sensor-data

This screenshot shows that when API mode is used, the frontend attempts to call /api/sensor-data. However, this route does not match the current backend dataset-based route structure, which creates an integration mismatch between frontend assumptions and backend availability.

new-frontend/frontend/src/hooks/useSensorData.js

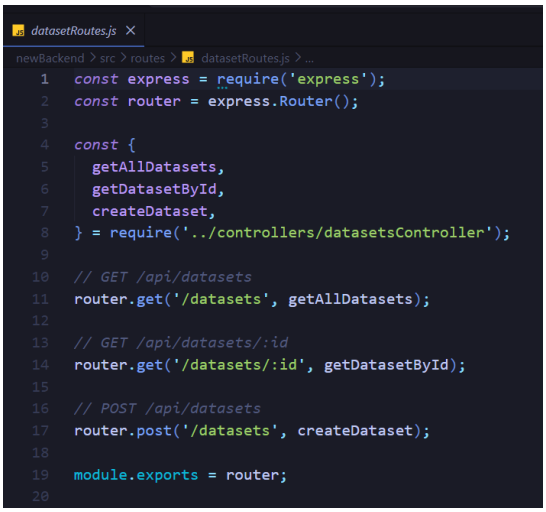
```
    fetch('/api/sensor-data')
      .then((res) => res.json())
      .then((json) => {
        setData(json);
        setLoading(false);
      })
      .catch((err) => {
        setError(err);
        setLoading(false);
      });
  }, [useMock]);

  return { data, loading, error };
};
```

Appendix C.4 - Backend uses dataset route structure

This screenshot shows that the backend provides dataset-based routes instead of the frontend's expected /api/sensor-data route. This supports the need for a documented API contract so the frontend team knows which backend endpoints should be used.

newBackend/src/routes/datasetRoutes.js



Appendix C.5 – Backend uses series route structure

This screenshot shows the backend route used to retrieve time-series data for a selected dataset. This confirms that the backend direction is based on dataset-specific series routes, rather than a single generic /api/sensor-data endpoint.

newBackend/src/routes/seriesRoutes.js

Appendix C.6 - Backend uses timestamp route structure

This screenshot shows that the backend also separates timestamp retrieval into a dataset-based route. This is important because the frontend may need timestamp data for filters, charts, or time range selection, so the contract should explain how this endpoint should be called.

newBackend/src/routes/timestampsRoutes.js

Appendix D: Backend Validation and Error Handling Evidence

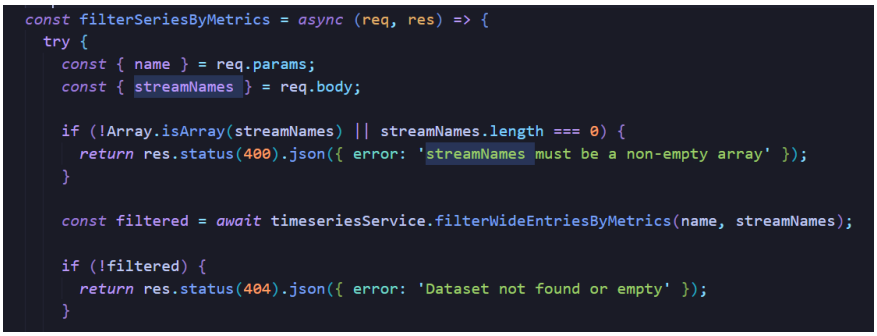
Appendix D shows backend validation and error handling evidence from the dataset-based backend path. The backend expects selected stream names to be sent using the request field streamNames when filtering dataset series. The current evidence shows that if this field is missing or empty, the backend returns a clear 400 Bad Request response. This supports the research recommendation that the frontend must follow a documented connection contract to avoid request format mismatches during integration. Full validation for stream names that do not exist in the selected dataset should be treated as a future improvement.

- newBackend/src/controllers/seriesController.js
- newBackend/src/controllers/datasetsController.js
- newBackend/src/controllers/timestampsController.js
- POST test for /api/datasets/:name/series/filter

Appendix D.1 - Capture missing/empty streamNames validation

This screenshot shows the backend filtering controller reading streamNames from the request body. It provides evidence that the frontend must send selected fields using the correct request field name when calling the filtering endpoint.

newBackend/src/controllers/seriesController.js



```
const filterSeriesByMetrics = async (req, res) => {
  try {
    const { name } = req.params;
    const { streamNames } = req.body;

    if (!Array.isArray(streamNames) || streamNames.length === 0) {
      return res.status(400).json({ error: 'streamNames must be a non-empty array' });
    }

    const filtered = await timeseriesService.filterWideEntriesByMetrics(name, streamNames);

    if (!filtered) {
      return res.status(404).json({ error: 'Dataset not found or empty' });
    }
```

Appendix D.2 - Dataset Controller Error Handling

This screenshot shows the backend error handling for the GET /api/datasets endpoint. If datasets cannot be loaded, the controller returns a clear 500 response with the message Failed to load datasets. This supports the API contract because the frontend needs to understand the expected backend error format when dataset loading fails.

newBackend/src/controllers/datasetsController.js

```
const datasetService = require('../services/datasetService');

/**
 * GET /api/datasets
 * Returns a list of all datasets.
 */
const getAllDatasets = async (req, res) => {
  try {
    const datasets = await datasetService.getAllDatasets();
    return res.status(200).json(datasets);
  } catch (err) {
    console.error('Error getting datasets:', err);
    return res.status(500).json({ error: 'Failed to load datasets' });
  }
};
```

Appendix D.3 - Open timestampsController.js

This screenshot shows backend error handling for timestamp-related requests. This is relevant because the frontend may use timestamp data for charts or filters, so it must handle backend error responses correctly.

newBackend/src/controllers/timestampsController.js

```
const getTimestampsForDatasetName = async (req, res) => {
  try {
    const { name } = req.params;

    const entries = await timeseriesService.getWideEntriesForDatasetName(name);

    if (!entries) {
      return res.status(404).json({ error: 'Dataset not found or empty' });
    }

    const timestamps = entries.map((e) => e.created_at);

    return res.status(200).json(timestamps);
  } catch (err) {
    console.error('Error getting timestamps:', err);
    return res.status(500).json({ error: 'Failed to load timestamps' });
  }
};
```

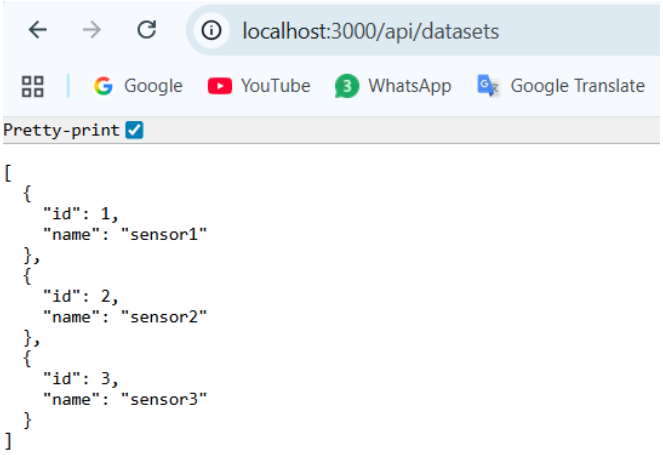
Appendix F: API Testing Evidence

Appendix F should include screenshots of Postman, Thunder Client, browser, terminal or test output showing successful and failed API calls.

- successful GET /api/dataset
- successful GET /api/datasets/:name/series
- successful GET /api/datasets/:name/timestamps
- failed request with missing or invalid streamNames
- successful POST /api/datasets/:name/series/filter

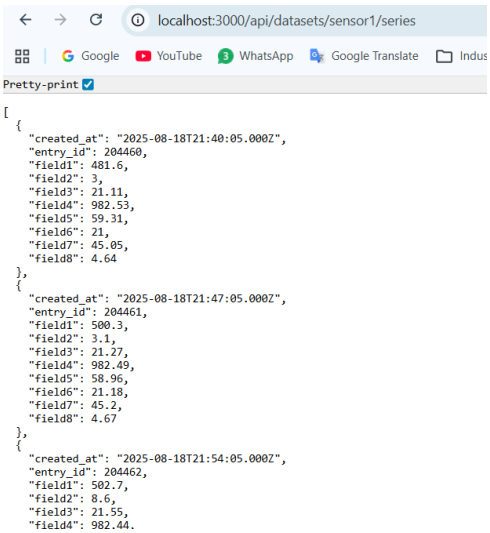
Appendix F.1 - GET /api/datasets

This screenshot shows a successful dataset list request. The backend returns available datasets: sensor1, sensor2, and sensor3.



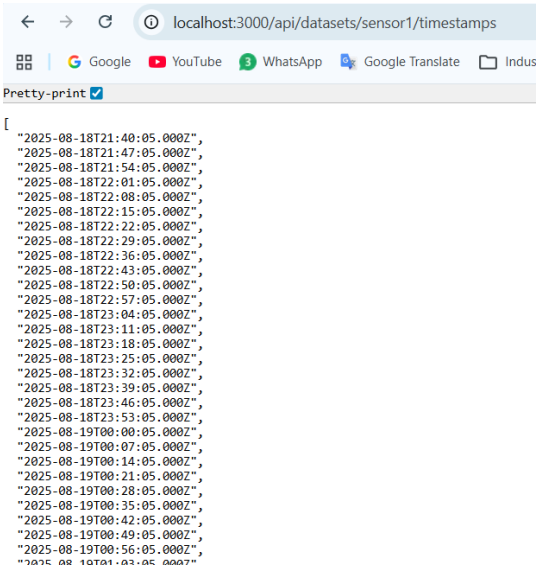
Appendix F.2 - GET /api/datasets/:name/series

This screenshot shows a successful request for sensor1 series data. The backend returns time-series records with timestamps and sensor fields.



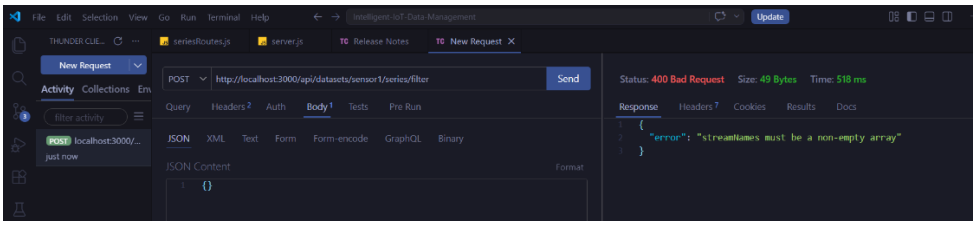
Appendix F.3 - GET /api/datasets/:name/timestamps

This screenshot shows a successful timestamp request for sensor1. The backend returns available timestamps that the frontend can use for time-based filtering.



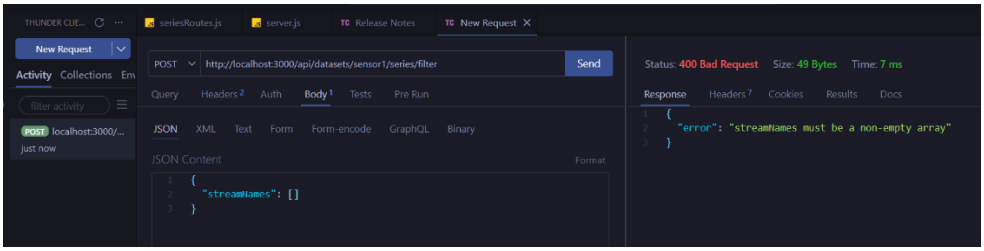
Appendix F.4 -Missing streamNames

This screenshot shows a POST request to /api/datasets/sensor1/series/filter with an empty request body {}. The backend returns 400 Bad Request with the error message streamNames must be a non-empty array. This proves that the backend validates the request body and that the frontend must send the required streamNames field.



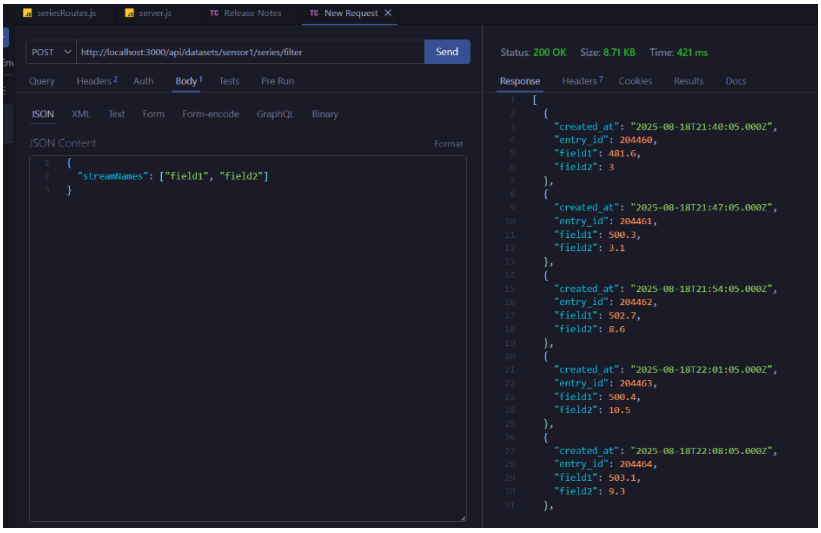
Appendix F.5 -Empty streamNames array

This screenshot shows a POST request where streamNames is provided but left as an empty array. The backend again returns 400 Bad Request, confirming that the frontend must send at least one selected stream name when filtering dataset series.



Appendix F.6-Successful filter request

This screenshot shows a successful POST request to /api/datasets/sensor1/series/filter using streamNames: ["field1", "field2"]. The backend returns 200 OK and provides filtered series data containing only the requested fields. This confirms that the filtering endpoint works when the frontend follows the expected request format.



References

Bogutskii V (2025) ‘Modern approaches to integrating frontend and backend systems in web applications’, *Scientific Research Journal*, 13(4):108–114, https://doi.org/10.31364/SCIRJ/v13.i04.2025.P04251024

Jayaprakash DB (2025) ‘Understanding API integration in modern enterprise applications’, *Sarcouncil Journal of Multidisciplinary*, 5(7):637–645, https://doi.org/10.5281/zenodo.16149567

MDN Web Docs (4 July 2025a) *GET request method*, MDN Web Docs, accessed 4 May 2026. https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods/GET

MDN Web Docs (4 July 2025b) *POST request method*, MDN Web Docs, accessed 4 May 2026. https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods/POST

Microsoft (22 August 2024) *REST API design guidance*, Microsoft Engineering Playbook, accessed 4 May 2026. https://microsoft.github.io/code-with-engineering-playbook/design/design-patterns/rest-api-design-guidance/

Microsoft Azure Architecture Center (8 May 2025) *Best practices for RESTful web API design*, Microsoft Learn, accessed 4 May 2026. https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design

OpenAPI Initiative (2024a) *OpenAPI Specification v3.1.1*, OpenAPI Initiative, accessed 4 May 2026. https://spec.openapis.org/oas/v3.1.1.html

OpenAPI Initiative (25 October 2024b) ‘Announcing OpenAPI Specification versions 3.0.4 and 3.1.1’, OpenAPI Initiative, accessed 4 May 2026. https://www.openapis.org/blog/2024/10/25/announcing-openapi-specification-patch-releases

React (2025) *Reusing logic with custom hooks*, React Documentation, accessed 4 May 2026. https://react.dev/learn/reusing-logic-with-custom-hooks